

ChessExplain 个人总结反思

王翟 2024011226

2026 年 6 月 3 日

1 项目缘起与定位

我是一名国际象棋半专业棋手，日常训练高度依赖引擎分析。无论是 Stockfish 还是 Lichess 的分析面板，它们都能给出精确的分数和推荐着法，但几乎从不解释“为什么”。接触过市面上大多数国际象棋平台后，我发现一个共同的痛点：引擎是一个黑盒——你看到结果，却看不到思考过程。

这学期的人工智能导论课程恰好涵盖了搜索算法、博弈树、启发式评估等内容。Alpha-Beta 剪枝和蒙特卡洛树搜索（MCTS）正是 Stockfish 等强引擎的核心技术。于是我萌生了一个想法：能否做一个平台，不追求最强的棋力，而是把 AI 的“思考过程”可视化出来，让棋手真正理解引擎为什么这样走？

这就是 ChessExplain 的起点——一个“可解释的国际象棋 AI 分析平台”。

2 白盒搜索：让算法透明

项目的技术核心是 Search Lab（搜索实验室），它实现了两种经典博弈搜索算法。

Alpha-Beta 剪枝是 Minimax 的工程优化版本。我的实现使用 `python-chess` 生成合法着法，递归搜索到指定深度，用 α 和 β 边界裁掉不可能影响结果的子树。为了让搜索结果可解释，我不仅返回最佳着法，还记录了搜索树的节点统计、剪枝位置和候选着法排序，前端用 ECharts 将整棵搜索树可视化呈现。

MCTS 遵循标准的四阶段流程：Selection（UCB1 选择子节点）、Expansion（扩展未访问着法）、Simulation（随机 rollout）、Backpropagation（回传结果）。实现中最棘手的 bug 是奖励回传时的视角翻转——MCTS 的 wins 统计到底代表白方胜率还是当前行棋方胜率？经过多次调试，我最终将所有分数统一为“白方视角”（正数白优、负数黑优），并在选择阶段根据行棋方转换偏好，这样 UI 显示和算法内部逻辑保持一致。

评估函数经历了三轮演进。最初使用纯子力评估（Material），但在开局浅深度（2-3 层）下所有候选着法都是 0 分，完全无法区分优劣——这对教学来说是致命的。后来加入 Piece-Square Table（位置表），再扩展到机动性、兵型和王安全等综合启发式，才让

用户在深度 2 就能看到有意义的评分差异。这个过程中我深刻体会到：一个 AI 系统的“正确性”和“可解释性”往往是两种不同的设计目标。

3 工程实战：从本地到云端

如果说算法实现是“上课学的”，那部署过程就是“课本上没有的”。

项目采用 React + Vite 前端、FastAPI + Python 后端的分层架构，部署在 Cloudflare Pages（静态前端）和 Railway（后端 API）上。看似简单的架构，实际踩遍了各种坑：

Stockfish 路径问题是最顽固的。本地 Windows 开发用 `engines/stockfish.exe` 没问题，但 Railway 的 Linux 容器里 stockfish 装在哪？我先后试过 Nixpacks 的 `nixPkgs`、`aptPkgs`、Dockerfile 直接下载二进制，甚至写了一个 `_find_stockfish()` 函数依次尝试七个候选路径。最后发现关键在于：Railway 的 Nixpacks 构建器需要用 `aptPkgs` 而非 `nixPkgs`（后者会切换到 Nix 基础镜像导致 Python 丢失），而且 `startCommand` 中的 `${PORT}` 变量不会自动展开。这些问题在本地永远遇不到，只有真正部署才暴露。

双后端目录是我的另一个教训。项目早期不知为何出现了 `backend/` 和 `phase2_research/backend/` 两个平行目录，代码已经分叉：一个缺少白盒引擎路由，一个没有智能路径搜索。直到 Railway 部署报 404 才发现根 `railway.toml` 指向的是 `phase2_research/backend/`，而 `backend/` 已经是僵尸代码。这是缺乏代码审查和配置管理的典型后果。

硬编码 URL 则是一个“能用就行”心态的典型案例。前端多处写了 `http://localhost:8000`，本地当然跑得通，一旦部署到生产环境就全部失效。后来统一抽取 `API_BASE` 配置、通过 `.env.production` 注入 Railway 域名才彻底解决。这个小问题教会我：环境差异不是“以后再说”的事，而是架构设计的第一天就要考虑的。

4 反思与收获

理论到实践的鸿沟比我预想的大得多。课堂上 Alpha-Beta 用伪代码几行就写完，但真正实现时要处理棋盘状态的深拷贝、合法着法生成、分数视角统一、搜索树结构的序列化传输，以及大深度下 UI 卡顿的用户体验问题。一个“能用”的 AI 系统，代码量 80% 花在算法之外。

AI 可解释性的价值在项目中得到了验证。当我看到搜索树可视化让棋手能够逐节点追溯引擎的思考路径时，我确信这比单纯输出一个分数更有教学意义。这也是我坚持“白盒引擎”而非直接调用 Stockfish 做搜索实验室的原因——强引擎给出答案，弱引擎解释过程。

工程习惯方面，`git worktree` 并行开发、`codemap` 文档维护、前后端测试覆盖（37 个前端测试 + `pytest` 后端测试）、`AbortController` 请求取消等实践，都是在项目推进中逐步建立起来的。

如果重来一次，我会在项目初期就确定统一的部署配置策略，避免双目录和硬编码 URL 这类返工；也会更早引入 AbortController，不让用户在长搜索中干等。

ChessExplain 不是一个棋力最强的引擎，但它做到了市面上大多数平台没做到的事：让 AI 搜索变得可见、可理解。这 500 行 Python 搜索代码和 2000 行前端交互逻辑，是我对“人工智能导论”课程最好的回答。